

Post-Quantum Cryptography

A Comprehensive Survey

Presentation Outline

Section	Topic
1	Introduction (<i>this document</i>)
2	Key Number-Theoretic Building Blocks in PQC
3	Current Work: NIST Standardisation and Main Families
4	Analysing PQC: Methods
5	Present Shortcomings
6	Future Prospects

Abstract

This document covers **Section 1: Introduction** of the group presentation on Post-Quantum Cryptography (PQC). We trace the evolution of cryptographic security from classical public-key systems, examine how Shor’s quantum algorithm invalidates their hardness assumptions, and introduce the landscape of post-quantum alternatives currently undergoing standardisation. The subsequent sections, presented by other team members, build directly on the foundations laid here.

0 Contents

1	Traditional Cryptosystems: An Overview	2
1.1	Symmetric vs. Asymmetric Cryptography	2
1.2	Hardness Assumptions Underlying Classical Security	2
2	RSA: The Cornerstone of Classical Public-Key Cryptography	2
2.1	Mathematical Foundations	3
2.2	Key Generation	3
2.3	Encryption and Decryption	3
2.4	Digital Signatures with RSA	3
2.5	Why RSA is Hard (Classically)	4
2.6	Parameter Sizes and Classical Security Levels	4
3	Shor’s Algorithm and the Collapse of Classical Hardness	4
3.1	Quantum Computing in Brief	4
3.2	The Core Insight: Factoring via Period Finding	4
3.3	Algorithm Sketch	5
3.4	Complexity Comparison	5
3.5	What Breaks and What Survives	5
3.6	The “Harvest Now, Decrypt Later” Threat	6
4	The Road Ahead: Introducing Post-Quantum Cryptography	6
4.1	What is Post-Quantum Cryptography?	6
4.2	Candidate Hard Problems in PQC	6
4.3	The NIST Standardisation Process	6
4.4	Transition Challenges	7
	Summary	7

1 Traditional Cryptosystems: An Overview

Modern secure communication rests on *asymmetric* (public-key) cryptography, first formalised by Diffie and Hellman in 1976. The central idea is elegant: two mathematically related keys are generated — a **public key** shared openly, and a **private key** kept secret — such that encrypting with one key can only be reversed with the other. Security does not rely on keeping the algorithm secret, only the private key.

1.1 Symmetric vs. Asymmetric Cryptography

Symmetric Cryptography

A single shared secret key is used for both encryption and decryption (e.g. AES, ChaCha20). It is fast and quantum-resistant with sufficiently large keys, but requires a *secure channel* to distribute the key — the classic “key exchange problem.”

Asymmetric / Public-Key Cryptography

Two linked keys are used. Security is based on the computational hardness of a mathematical problem. Solves the key-distribution problem but is orders of magnitude slower than symmetric ciphers.

In practice, modern protocols (TLS, SSH, PGP) use *hybrid* schemes: asymmetric cryptography securely establishes a shared secret, which then seeds a fast symmetric cipher for bulk data encryption. **Defeating the asymmetric layer therefore breaks the entire session.**

1.2 Hardness Assumptions Underlying Classical Security

Three mathematical problems have historically underpinned public-key cryptography:

- (1) **Integer Factorisation Problem (IFP):** Given a large composite $N = p \cdot q$ with p, q large primes, recover p and q . This is the foundation of RSA.
- (2) **Discrete Logarithm Problem (DLP):** Given g, p , and $y = g^x \bmod p$, recover x . This underpins classical Diffie–Hellman and the Digital Signature Algorithm (DSA).
- (3) **Elliptic Curve Discrete Logarithm Problem (ECDLP):** Given points P and $Q = kP$ on an elliptic curve, recover the scalar k . This is the basis of ECDH and ECDSA, offering comparable security to RSA with much smaller key sizes.

All three problems are *believed* to be classically hard (sub-exponential or better attacks are known, but remain impractical for large parameters). As we shall see in section 3, a sufficiently powerful quantum computer can solve all three in polynomial time.

2 RSA: The Cornerstone of Classical Public-Key Cryptography

RSA, introduced by Rivest, Shamir, and Adleman in 1977, is arguably the most widely deployed public-key cryptosystem in history. It appears in TLS certificates, digital signatures, secure email, and countless embedded systems. Understanding RSA concretely is essential for appreciating why quantum computing is so disruptive.

2.1 Mathematical Foundations

RSA relies on two elementary results from number theory.

Theorem 2.1 (Fermat's Little Theorem). *If p is prime and $\gcd(a, p) = 1$, then $a^{p-1} \equiv 1 \pmod{p}$.*

Definition 2.1 (Euler's Totient Function). For $n = p \cdot q$ with distinct primes p and q :

$$\phi(n) = (p-1)(q-1).$$

Any integer a with $\gcd(a, n) = 1$ satisfies $a^{\phi(n)} \equiv 1 \pmod{n}$.

2.2 Key Generation

Step 1. Choose two large, distinct primes p and q (each ≥ 2048 bits in modern deployments).

Step 2. Compute the modulus $n = p \cdot q$.

Step 3. Compute $\phi(n) = (p-1)(q-1)$.

Step 4. Choose a public exponent e such that $1 < e < \phi(n)$ and $\gcd(e, \phi(n)) = 1$. Commonly $e = 65537 = 2^{16} + 1$.

Step 5. Compute the private exponent d satisfying $e \cdot d \equiv 1 \pmod{\phi(n)}$ via the extended Euclidean algorithm.

Key Pair Summary

Public key: (n, e) — shared with everyone.

Private key: (n, d) — kept secret.

The primes p and q must be *destroyed* after key generation.

2.3 Encryption and Decryption

Given a plaintext message m represented as an integer $0 \leq m < n$:

$$\textbf{Encryption: } c = m^e \bmod n$$

$$\textbf{Decryption: } m = c^d \bmod n$$

Correctness. By Euler's theorem, $c^d = m^{ed} = m^{1+k\phi(n)} = m \cdot (m^{\phi(n)})^k \equiv m \pmod{n}$.

2.4 Digital Signatures with RSA

The asymmetry of the key pair also enables digital signatures:

$$\textbf{Sign: } \sigma = \mathcal{H}(m)^d \bmod n \quad \textbf{Verify: } \sigma^e \bmod n \stackrel{?}{=} \mathcal{H}(m)$$

where $\mathcal{H}$ is a cryptographic hash function (e.g. SHA-256). Only the holder of d can produce a valid signature, but anyone with e can verify it.

2.5 Why RSA is Hard (Classically)

Hard Problem 2.1 (Integer Factorisation). *Given $n = p \cdot q$, factor n into p and q .*

An adversary who can factor n immediately recovers $\phi(n)$ and hence d . The best known classical algorithm for factoring is the **General Number Field Sieve (GNFS)**, with sub-exponential complexity:

$$L_n\left[\frac{1}{3}, \sqrt[3]{\frac{64}{9}}\right] = \exp\left(\left(\sqrt[3]{\frac{64}{9}} + o(1)\right) (\ln n)^{1/3} (\ln \ln n)^{2/3}\right).$$

For n of 2048 bits, this remains computationally infeasible on any foreseeable classical hardware. This sub-exponential hardness is precisely the gap that a quantum computer closes.

2.6 Parameter Sizes and Classical Security Levels

RSA Modulus (bits)	Symmetric Equivalent	Status
512	~56 bits	Broken (1999)
1024	~80 bits	Deprecated
2048	~112 bits	Current minimum
3072	~128 bits	Recommended
4096	~140 bits	High-security use

Even at 4096 bits, RSA provides *zero* security against a cryptographically relevant quantum computer running Shor’s algorithm.

3 Shor’s Algorithm and the Collapse of Classical Hardness

3.1 Quantum Computing in Brief

A classical computer operates on *bits* (0 or 1). A quantum computer operates on *qubits*, which exploit two quantum phenomena:

- **Superposition:** A qubit can exist in a continuum of states $\alpha|0\rangle + \beta|1\rangle$ (with $|\alpha|^2 + |\beta|^2 = 1$), encoding both values simultaneously until measured.
- **Entanglement and Interference:** Multiple qubits can be correlated such that quantum gates amplify probability amplitudes for correct answers while cancelling those for incorrect ones.

An n -qubit register can represent 2^n states simultaneously. For certain structured problems, quantum algorithms exploit this parallelism to achieve exponential speedups.

3.2 The Core Insight: Factoring via Period Finding

Shor’s algorithm (1994) reduces integer factorisation to a *period-finding* problem, which quantum computers can solve efficiently using the **Quantum Fourier Transform (QFT)**.

Reduction: Factoring $\rightarrow$ Period Finding

To factor n , choose a random a with $\gcd(a, n) = 1$ and find the *order* (period) r of a modulo n :

$$r = \min_{r>0} \{a^r \equiv 1 \pmod{n}\}.$$

If r is even and $a^{r/2} \not\equiv -1 \pmod{n}$, then

$$\gcd(a^{r/2} \pm 1, n)$$

yields a non-trivial factor of n with high probability.

3.3 Algorithm Sketch

1. Choose $a \in \{2, \dots, n-2\}$ uniformly at random.
2. Check $\gcd(a, n)$. If > 1 , a factor is found immediately (classical step).
3. Prepare a quantum superposition over a register of size $\sim 2 \log_2 n$ qubits.
4. Apply the **Quantum Fourier Transform** to extract the period r of the function $f(x) = a^x \bmod n$.
5. Use classical post-processing to obtain a non-trivial factor of n .

3.4 Complexity Comparison

Algorithm	Type	Complexity
Trial Division	Classical	$O(\sqrt{n})$
GNFS (best classical)	Classical	$L_n[1/3, c]$ (sub-exponential)
Shor's Algorithm	Quantum	$O((\log n)^2 \log \log n \cdot \log \log \log n)$

Shor's algorithm runs in **polynomial time** in the number of bits of n . This represents an *exponential* speedup over the best classical algorithm. For a 2048-bit RSA modulus, GNFS would require time estimated at millions of computer-years; Shor's algorithm on a sufficiently powerful quantum computer could solve it in hours or less.

3.5 What Breaks and What Survives**Cryptographic Impact of Shor's Algorithm****Broken (polynomial-time quantum attack):**

- RSA (relies on IFP)
- Classical Diffie–Hellman and DSA (rely on DLP)
- Elliptic Curve Cryptography: ECDH, ECDSA (rely on ECDLP)

Only weakened (Grover's Algorithm gives quadratic speedup):

- Symmetric ciphers (AES-256 remains secure; AES-128 is borderline)

- Cryptographic hash functions (SHA-256 remains secure with doubled output)

3.6 The “Harvest Now, Decrypt Later” Threat

Even without a cryptographically relevant quantum computer today, adversaries can store encrypted traffic now and decrypt it retrospectively once quantum hardware matures. This *harvest-now, decrypt-later* attack means that data requiring long-term confidentiality — government communications, medical records, financial transactions — is **already at risk** and migration must begin now.

4 The Road Ahead: Introducing Post-Quantum Cryptography

4.1 What is Post-Quantum Cryptography?

Post-Quantum Cryptography (PQC) refers to cryptographic algorithms designed to be secure against both classical *and* quantum adversaries. The algorithms are intended to run on *classical hardware* — no quantum computer is required to use them — but their underlying mathematical problems are believed to resist known quantum attacks.

Core Design Goal of PQC

Replace IFP, DLP, and ECDLP as hard problems with alternative problems for which no polynomial-time quantum algorithm is currently known, while maintaining practical efficiency on existing hardware.

4.2 Candidate Hard Problems in PQC

The PQC landscape is organised around several distinct families of hard problems, each forming the basis for one or more cryptographic constructions. These will be explored in depth in **Section 2** by the next presenter.

Family	Hard Problem	Representative Schemes
Lattice-based	SVP, LWE, MLWE, SIS	CRYSTALS-Kyber, CRYSTALS-Dilithium
Code-based	Syndrome Decoding	Classic McEliece, BIKE, HQC
Hash-based	Preimage / collision resistance	XMSS, LMS, SPHINCS ⁺
Multivariate	MQ-polynomial solving	Rainbow, GeMSS
Isogeny-based	CSSI, SSDDH	SIKE (broken 2022), SQIsign

4.3 The NIST Standardisation Process

In 2016, the US National Institute of Standards and Technology (NIST) launched a global competition to standardise PQC algorithms, following the same model as the AES and SHA-3 competitions. After multiple rounds of public cryptanalysis:

- **FIPS 203** (August 2024): ML-KEM (CRYSTALS-Kyber) — Key Encapsulation

- **FIPS 204** (August 2024): ML-DSA (CRYSTALS-Dilithium) — Digital Signatures
- **FIPS 205** (August 2024): SLH-DSA (SPHINCS⁺) — Digital Signatures
- **FIPS 206** (in progress): FN-DSA (FALCON) — Digital Signatures

The NIST process, its evaluation criteria, and the main algorithmic families will be covered in detail in **Section 3**.

4.4 Transition Challenges

Migrating from classical to post-quantum cryptography is not merely a matter of swapping keys. Key challenges include:

- (1) **Larger key and ciphertext sizes:** PQC schemes typically have significantly larger public keys and ciphertexts than RSA or ECC, imposing bandwidth and storage costs.
- (2) **Performance:** Some PQC schemes are computationally heavier, particularly on constrained devices (IoT, smart cards).
- (3) **Hybrid deployment:** During the transition period, combining classical and PQC schemes (hybrid cryptography) ensures security under both threat models.
- (4) **Protocol integration:** Existing standards (TLS, SSH, X.509, PKIX) require updates; this is an active engineering effort.
- (5) **Cryptographic agility:** Systems must be designed to swap algorithms without full rewrites, a lesson learned from the DES → AES migration.

Sections 4 and 5 of this presentation will analyse PQC schemes in detail and examine current shortcomings in the transition effort. Section 6 will conclude with future prospects and open research questions.

4 Summary of This Section

This introduction has established the foundations for the rest of the presentation:

1. Classical public-key cryptography (RSA, DH, ECC) derives its security from the computational hardness of IFP, DLP, and ECDLP on classical machines.
2. RSA, examined in detail, encapsulates all the core mechanisms: modular arithmetic, trapdoor one-way functions, and hardness reductions.
3. Shor's quantum algorithm (1994) solves the order-finding problem in polynomial time, collapsing the security of all three classical hard problems simultaneously.
4. The *harvest now, decrypt later* attack model means the threat is present even before large-scale quantum computers exist.
5. Post-Quantum Cryptography offers a family of alternative hard problems — based on lattices, codes, hash functions, and more — which remain quantum-resistant as far as is currently known.

6. NIST has completed its first standardisation round (2024), providing concrete algorithmic choices for the community.

The next presenter will cover the key number-theoretic building blocks that underlie the PQC families introduced above.

4 References

- [1] R. Rivest, A. Shamir, and L. Adleman, “A Method for Obtaining Digital Signatures and Public-Key Cryptosystems,” *Communications of the ACM*, vol. 21, no. 2, pp. 120–126, 1978.
- [2] P. W. Shor, “Algorithms for Quantum Computation: Discrete Logarithms and Factoring,” *Proceedings of the 35th Annual Symposium on Foundations of Computer Science*, pp. 124–134, 1994.
- [3] NIST, “Post-Quantum Cryptography Standardization,” <https://csrc.nist.gov/projects/post-quantum-cryptography>, 2024.
- [4] D. J. Bernstein and T. Lange, “Post-Quantum Cryptography,” *Nature*, vol. 549, pp. 188–194, 2017.
- [5] W. Diffie and M. E. Hellman, “New Directions in Cryptography,” *IEEE Transactions on Information Theory*, vol. 22, no. 6, pp. 644–654, 1976.